

Contenido

01

¿Por qué
versionar una
API?

02

Versionamiento
Semántico

03

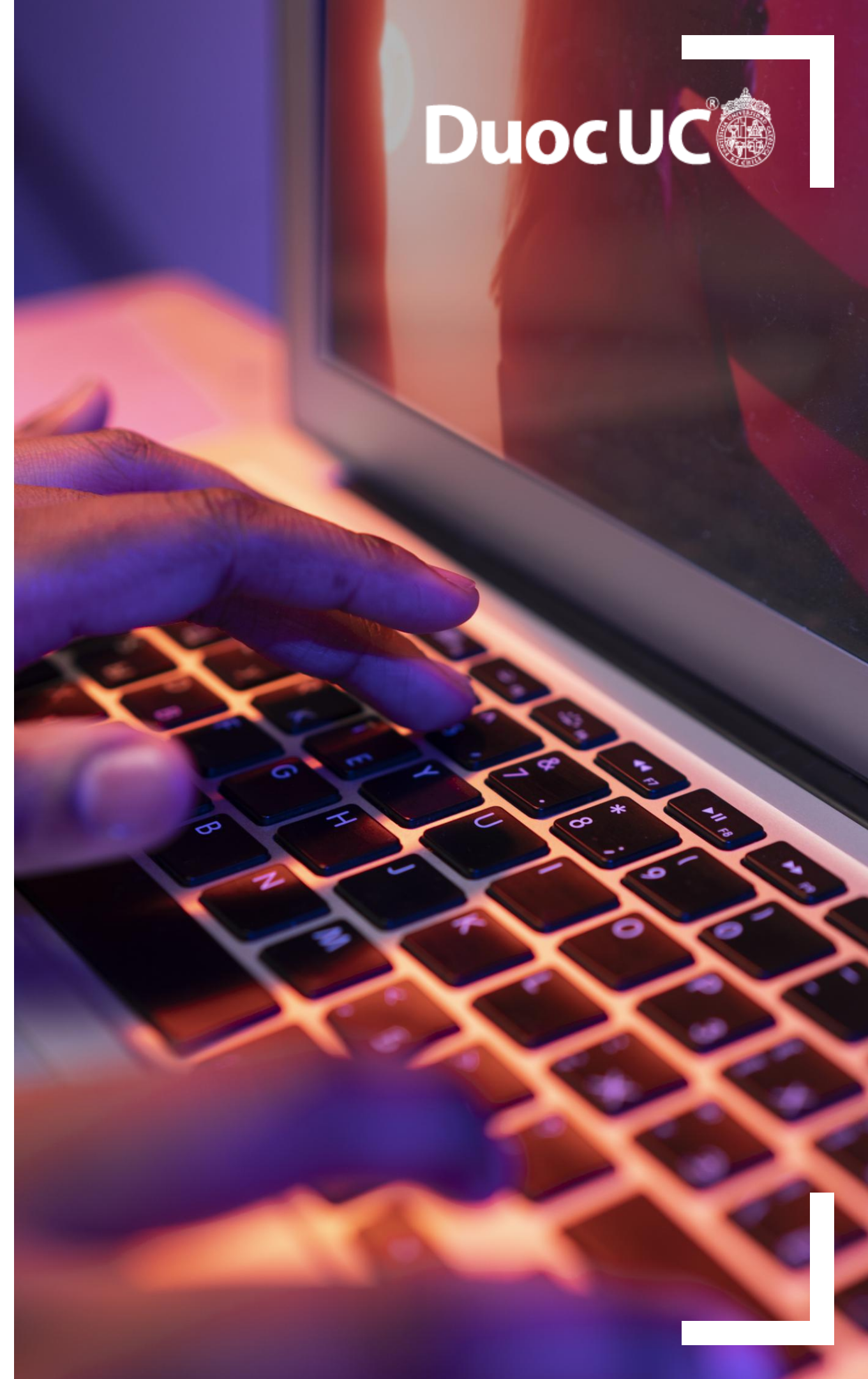
Versionamiento Api-
Gateway

04

Proceso de
Deprecación de una
API

05

Buenas Practicas



06

¿Por qué versionar una API?



¿Por qué versionar una API?

- Las API cambian con el tiempo, pero no puedes romper a todos los clientes en cada cambio.
- Permite agregar nuevas funcionalidades sin romper las anteriores.
- Corrige errores sin afectar clientes existentes.
- Migra modelos de datos gradualmente.
- Mantén varias versiones durante actualizaciones.
- Controla la deprecación y migración de clientes.

Metáfora: Calles y direcciones

Una API es como una **ciudad**. Los clientes esperan que una calle lleve siempre al mismo lugar.

Si cambias toda la ciudad sin avisar, nadie sabrá llegar.

Versionar es poner señales: "Esta es la ciudad v1", "Aquí comienza v2".
Los clientes deciden cuándo migrar, no tú.



Tipos de versionamiento de APIs

Enfoque	Ejemplo	Ventajas	Usado por
URL	https://api.miapp.com/v1/usuarios	Fácil de entender y documentar, ideal para API Gateway	AWS, Google, Stripe
Headers	Accept: application/vnd.miapp.v1+json	URLs limpias, flexible en APIs internas	GitHub
Query Param	/usuarios?version=2	Sencillo, pero no respeta semántica y es confuso	Casos puntuales

¿Cuándo crear una nueva versión?

Incompatibles REQUIERE NUEVA VERSIÓN

- Cambiar estructura del JSON
- Eliminar campos existentes
- Cambiar comportamiento de endpoints
- Cambiar reglas de negocio
- Remover recursos completos

Compatibles NO REQUIERE VERSIÓN

- + Agregar campos nuevos
- + Mejorar performance
- + Agregar endpoints nuevos
- + Correcciones menores (Bugs)
- + Cambios cosméticos



REGLA DE ORO

Nunca rompas a tus clientes.
Versiona solo al romper compatibilidad.

¿Por qué el API Gateway es clave en el versionamiento?

El API Gateway se encuentra frente a todos los clientes (front, móviles, terceros, microservicios externos). Es la “puerta principal” del sistema. Por eso, versionar APIs en el Gateway permite: mantener múltiples versiones funcionando al mismo tiempo, protegerlas individualmente, monitorearlas por separado, hacer rollouts controlados sin afectar a los clientes actuales. En pocas palabras:

👉 **El API Gateway es el único lugar donde puedes versionar sin romper a los demás sistemas.**



Versionamiento en un API Gateway

Gestión mediante **Stages (Escenarios)** independientes que permiten convivencia sin interferencias.

DEV

QA

BETA

V1

V2

PROD

CLIENTES ANTIGUOS

Legacy

GET /v1/usuarios

**Autenticación Simple**

API Key estándar

**Límites (Throttling)**

Bajo / Estricto

**Backend Monolito**

Integración Legacy / On-Premise

NUEVAS IMPLEMENTACIONES

Modern

GET /v2/usuarios

**Seguridad Avanzada**

OAuth2 / OIDC / JWT

**Optimización de Tráfico**

Control de Bursts / Rate Limits altos

**Backend Moderno**

Microservicios / Serverless / Nueva DB



¿Por qué gestionar versiones independientes?



Aislamiento Total

Cada versión tiene sus propias reglas de negocio, caché y seguridad. Modificar V2 no rompe V1.



Observabilidad Granular

Logs, métricas y alertas separadas. Sabes exactamente qué versión genera errores o latencia.



Evolución del Backend

Permite apuntar a arquitecturas completamente distintas (Serverless vs Legacy) bajo el mismo dominio.

Estrategia: Versionamiento Semántico

El estándar universal (SemVer) para comunicar el impacto de los cambios a través del número de versión.

1 . 0 . 0

MAJOR



v1 → v2

Cambios Incompatibles

Indica cambios que rompen la API (breaking changes). Los clientes deben actualizar su código para seguir funcionando.

MINOR



v1.0 → v1.1

Nuevas Funciones

Agrega funcionalidad de manera retro-compatible. Los clientes antiguos siguen funcionando sin problemas.

PATCH



v1.1.0 → v1.1.1

Correcciones / Fixes

Corrección de errores internos que no afectan la interfaz pública. Seguro de actualizar automáticamente.

Proceso de Deprecación de una API

1. Aviso de Deprecación

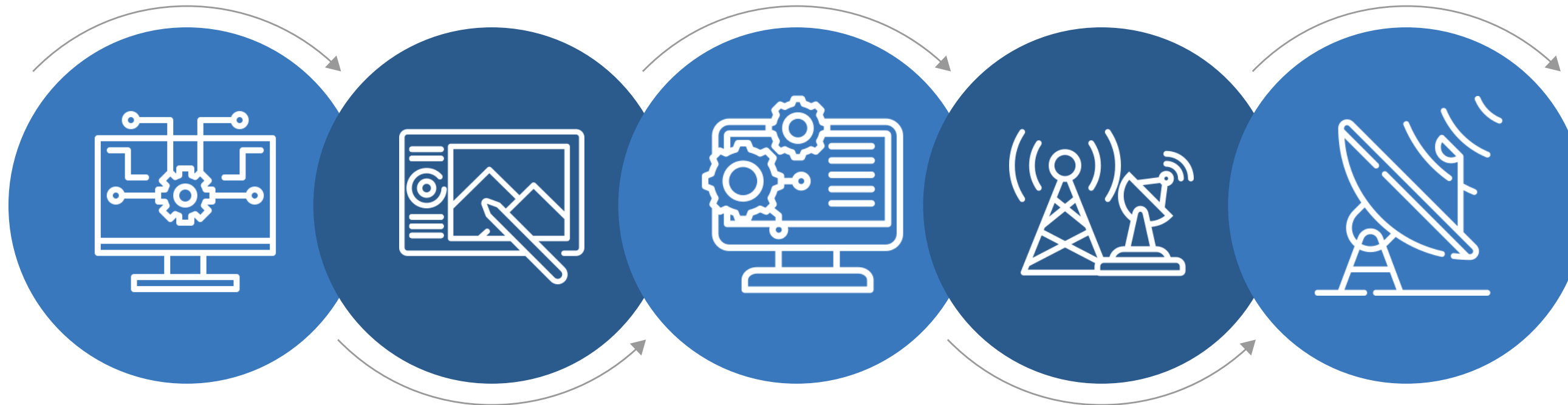
informa oficialmente que una versión será retirada.
Documentación, emails y headers como:
Deprecation: true · Sunset: 2025-12-01.

3. Coexistencia de Versiones

Mantén /v1 y /v2 activas durante meses.
Permite migración gradual sin afectar clientes existentes.

5. Apagado Controlado

Deshabilita progresivamente rutas antiguas.
Redirige tráfico, bloquea accesos y elimina la versión sin impactar producción.



2. Comunicación a Integradores

Notifica a equipos internos y externos.
Incluye fecha límite, guía de migración y cambios incompatibles (breaking changes).

4. Monitoreo y Uso Residual

Supervisa tráfico, errores y consumo por versión.
Define cuándo es seguro retirar la versión antigua.

Buenas Prácticas Profesionales

Estándares de oro para el diseño y mantenimiento de APIs



Versionamiento por URL

En APIs públicas, usa rutas claras como /v1/recurso para facilitar el consumo.



Compatibilidad Retroactiva

Nunca rompas clientes existentes. Si es necesario, crea una nueva versión mayor.



Documentación Clara

Mantén changelogs y especificaciones (OpenAPI) actualizadas por versión.



Rutas Separadas

Aísla el tráfico. v1 y v2 deben vivir independientemente en el Gateway.



Comportamiento Inmutable

No cambies la lógica de una versión publicada. Si cambia, es una versión nueva.



Automatización

Automatiza el ciclo de vida: deployment, staging y políticas de deprecación.

Felicitaciones
¡Has finalizado el módulo!

Links de interés

Semantic Versioning 2.0.0 – SemVer Oficial

<https://semver.org/>

IETF – HTTP Deprecation Header (RFC 8594)

<https://www.rfc-editor.org/rfc/rfc8594>

AWS API Gateway – Working with API Versions

<https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-api-versioning.html>

Azure API Management – Versioning in API Management

<https://learn.microsoft.com/azure/api-management/api-management-versions>

DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO

 Comisión Nacional
de Acreditación
CNA-Chile

NIVEL DE EXCELENCIA
HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la
calidad / Vinculación con el Medio / Investigación, creación y/o innovación